Pac-Man on Cmod A7 — Build Plan

Oct 17, 2026 · @Adrian Gosko

Overview

You'll build this in 11 phases, each ending in something you can see on the monitor: the first win is a colour test pattern, the last is a playable game with four ghosts that boots straight from flash.

Item	Choice	Note
Board	Cmod A7-35T (XC7A35T-1CPG236C)	12 MHz oscillator, 2 buttons, 2 LEDs, 1 Pmod port, 48 breadboard pins
Video out	Digilent Pmod VGA	4 bits per colour channel = 4,096 colours
Video mode	640x480 @ 60 Hz	25.175 MHz pixel clock; your 1080p monitor upscales it
Input	4 push buttons on breadboard pins	Up, down, left, right
Language and tools	Verilog-2001 in Vivado (free Standard edition)	Simulate in Vivado's built-in simulator (xsim)
Sound (late phase)	Piezo or small speaker driven by PWM	The Cmod A7 has no audio output

Three design decisions that shape everything else

- **No framebuffer.** A full 640x480 frame at 12 bits is about 3.7 Mbit, roughly double the 1.8 Mbit of block RAM in the XC7A35T. Instead you work out each pixel's colour on the fly as the beam scans, from a tile map plus sprite images. This is "racing the beam", which is how the original arcade hardware worked too.
- **Tiles at 2x scale.** Draw tiles as 8x8 pixel art and display each pixel as 2x2, so a tile is 16x16 on screen. A 28-wide by 30-tall maze then fills exactly 448x480, leaving a 192-pixel panel on the right for score, lives and level. Sprites are 16x16 art, shown at 32x32.

- **One clock, two rhythms.** Everything runs on the 25.175 MHz pixel clock. The renderer works every pixel; game logic (movement, ghosts, collisions) runs once per frame, triggered by a one-cycle `frame_tick` pulse at the start of vertical blanking. That avoids tearing and avoids clock-domain-crossing bugs.

Draw your own sprite and maze art rather than ripping the arcade ROM graphics; it's more honest for a portfolio piece and the art is simple.

Hardware setup and wiring

The catch: the Pmod VGA uses two 12-pin connectors (14 signals), but the Cmod A7 has only one Pmod port with 8 I/O. So one half plugs into the Pmod port and the other half is jumper-wired from the breadboard pins (Cmod A7 manual, Pmod VGA page).

Shopping list

- Pmod VGA, VGA cable, micro-USB cable
- Full-size breadboard (the Cmod A7 sits across the centre channel)
- 4 tactile push buttons
- Male-to-female and male-to-male jumper wires
- Later: a piezo buzzer or small 8 Ω speaker with a transistor, for sound

Wiring map

Pmod VGA pin	Signal	Goes to
J1 pins 1-4	R0-R3 (red)	Cmod Pmod port JA pins 1-4
J1 pins 7-10	B0-B3 (blue)	Cmod Pmod port JA pins 7-10
J1 pins 5, 6, 11, 12	GND, 3.3 V	JA's own GND and 3.3 V pins (automatic when plugged in)
J2 pins 1-4	G0-G3 (green)	4 free Cmod breadboard pins, via jumpers
J2 pin 7, pin 8	HSYNC, VSYNC	2 free Cmod breadboard pins, via jumpers
J2 pins 5, 11	GND	Breadboard GND rail (Cmod pin 25)

The J1 pinout is from Digilent's page. The J2 assignment above (green on 1-4, syncs on 7-8) is from memory, so confirm it against the Pmod VGA reference manual before powering up. If the Pmod VGA can't physically sit in JA because of the breadboard, jumper-wire J1 too.

Buttons

Wire each button between a free breadboard pin and the GND rail, and enable the FPGA's internal pull-up in the constraints file. That makes the buttons active-low (pressed = 0) and saves needing a 3.3 V rail or external resistors. Use the two on-board buttons as reset (BTN0) and start/pause (BTN1).

Pins to avoid

- Pins 15 and 16: these are the two analog inputs
- Pin 24 (VU, external power in) and pin 25 (GND)

Constraints file

Download `Cmod-A7-Master.xdc` from Digilent's `digilent-xdc` GitHub repo, uncomment only the lines for pins you use, and rename the ports to match your top module. A button line with the pull-up looks like this (take the real package pin from the master file):

```
set_property -dict { PACKAGE_PIN <pin> IOSTANDARD LVCMOS33 PULLUP true }
[get_ports { btn_up_n }];
```

The Vivado workflow

Every feature goes round the same loop: write a module, prove it in simulation, and only then build and program the board. Simulation takes seconds and shows every signal; a hardware build takes minutes and shows you only the screen.

Folder layout (keep it in git)

```
pacman/
  rtl/      top.v, vga_timing.v, renderer.v, ... (design sources)
  sim/      tb_vga_timing.v, tb_top.v, ... (testbenches)
  mem/      maze.mem, tiles.mem, sprites.mem, font.mem (ROM contents)
  constr/   cmod_a7.xdc
  tools/    png_to_mem.py (converts your pixel art to .mem files)
  vivado/   the Vivado project (add to .gitignore; it's regenerable)
```

Keep sources outside the Vivado project folder and add them to the project by reference, so the project can be deleted and rebuilt at any time.

The loop, step by step

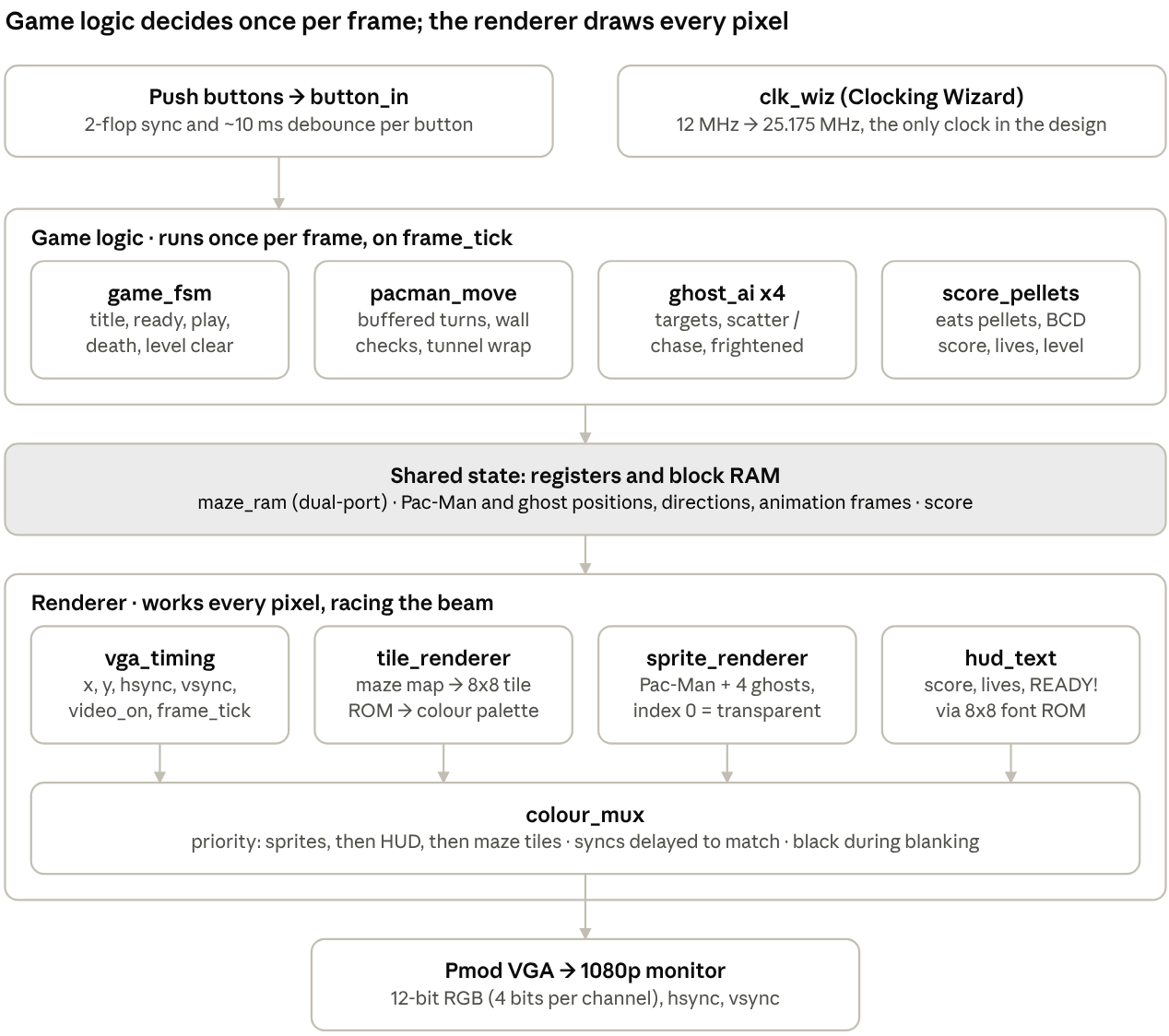
1. **Create the project once.** New RTL project, part `xc7a35tcpg236-1` (or install Digilent's board files and pick Cmod A7-35T). Add `rtl/`, `sim/` (as simulation-only sources), `mem/` and `constr/`.
2. **Write the module** in `rtl/`. Clocked logic uses `always @(posedge clk)` with `<=`; combinational logic uses `always @*` with `=`.
3. **Write a testbench** in `sim/` that drives a clock and inputs, and checks outputs with `if (...) $display("FAIL ...")`. Set it as the simulation top.
4. **Run Behavioral Simulation.** Read the waveform: counters wrapping at the right values, pulses one cycle wide, no red X (undefined) signals.
5. **Run Synthesis.** Read every warning. "Inferred latch", "multi-driven net" and "signal is unconnected" almost always mean a bug.
6. **Run Implementation**, then open the timing summary. Worst Negative Slack (WNS) must be zero or positive; negative means the logic is too slow for the clock.
7. **Generate Bitstream**, open Hardware Manager, auto-connect, Program Device. This is volatile: it's gone when the board loses power.
8. **When a phase is finished,** write the bitstream to the on-board Quad-SPI flash so the board boots your game at power-up (Digilent's Cmod A7 programming guide covers the flash part to select).

The trick that saves the most time: simulate the screen

In your top-level testbench, write every visible pixel of one frame to a `.ppm` image file with `$fopen` / `$fwrite`. PPM is a plain-text image format: a short header (`P3 640 480 15`) then one "r g b" triple per pixel. Open the file in GIMP or an online viewer and you see exactly what the monitor would show, without building anything. Simulate 2-3 frames to check animation and movement too.

Architecture

The design splits into a slow half that decides what's happening and a fast half that draws it, joined by a small shared state.



module architecture · inputs, game logic, shared state, renderer, output

Each box is one Verilog file in `rtl/`, and `top.v` wires them together. The renderer never changes game state, and game logic never touches pixels, so you can test each half on its own.

Phased milestones

Start at Phase 0 and don't skip ahead: each phase is built on, and debugged with, the one before. Phases 0-6 get you the animated, steerable Pac-Man eating dots; 7-10 turn it into the arcade game.

Phase 0 — Toolchain and blinky

- [ ] Install Vivado (free Standard edition covers the Artix-7 35T) and Digilent's board files
- [ ] Blink an LED at 1 Hz with a counter on the 12 MHz clock
- [ ] Program over JTAG, then try writing it to flash and power-cycling

Done when: the LED blinks, and you've been round the whole loop from the Vivado workflow section once.

Phase 1 — VGA timing and a test pattern

- [ ] Clocking Wizard IP: 12 MHz in, 25.175 MHz out (25.000 MHz also works on nearly every monitor)
- [ ] `vga_timing.v`: horizontal count 0-799 (640 visible, 16 front porch, 96 sync, 48 back porch), vertical count 0-524 (480 visible, 10 front porch, 2 sync, 33 back porch); both sync pulses active-low
- [ ] Outputs: `hsync`, `vsync`, `video_on`, `x`, `y`, and a one-cycle `frame_tick` when y reaches 480
- [ ] Testbench checks the counts and sync widths, and dumps a PPM frame
- [ ] Draw 8 vertical colour bars; force RGB to 0 whenever `video_on` is low

Done when: the monitor shows the bars and its info menu reports 640x480 at 60 Hz.

Phase 2 — Buttons and a moving square

- [ ] `button_in.v`: two flip-flops to synchronise each button, then a debounce counter (about 10 ms, roughly 252,000 cycles at 25.175 MHz)
- [ ] Move a 16x16 white square 1 pixel per `frame_tick` while a button is held

Done when: the square moves smoothly in all four directions with no jitter or double-steps.

Phase 3 — Tile renderer and a static maze

- [ ] Design about 32 tiles of 8x8 art: blank, wall corners and edges, pellet, power pellet, ghost-house door
- [ ] Write the 28x30 maze as a tile-index map in `maze.mem`, loaded with `$readmemh`
- [ ] Renderer pipeline: pixel position → tile column and row (divide by 16, i.e. drop 4 bits) → read tile index from the map → read the tile's pixel → palette → RGB

- [ ] Block RAM reads take a clock cycle each, so delay `hsync`, `vsync` and `video_on` by the same number of cycles as the pixel pipeline

Done when: the full maze shows, sharp-edged, with no shifted columns along the left edge.

Phase 4 — Animated Pac-Man sprite

- [ ] Draw 3 frames of 16x16 art (mouth closed, half, open), facing right
- [ ] Get the other 3 directions for free by swapping or mirroring the sprite ROM's x and y address bits, so you only store one direction
- [ ] Sprite renderer: if the current pixel is inside Pac-Man's 32x32 box, read his pixel; colour index 0 = transparent, so the maze shows through
- [ ] Animation counter: advance the frame every 4 frame ticks while moving, cycling closed → half → open → half
- [ ] Direction register driven by the last button pressed

Done when: Pac-Man sits in the maze, chomping, and turns to face whichever button you press.

Phase 5 — Movement and wall collision

- [ ] Store position in maze pixels (native 8x8-tile units, not screen pixels) and update only on `frame_tick`
- [ ] Allow a turn only when Pac-Man is centred on a tile and the next tile that way isn't a wall
- [ ] Buffer the wanted direction: if you press Up just before a junction, he turns when he gets there (this is what makes it feel like Pac-Man)
- [ ] Stop at walls and freeze the mouth animation while stopped
- [ ] Tunnel: wrap from the left edge to the right and back

Done when: you can drive Pac-Man round the whole maze without clipping into a wall.

Phase 6 — Pellets, score and HUD

- [ ] Turn the maze map into dual-port RAM: game logic writes, the renderer reads
- [ ] When Pac-Man reaches a pellet tile's centre, write blank there and add 10 (power pellet: 50)

- [ ] Keep the score in BCD (one 4-bit digit per decimal place) so you can display it without dividing by 10
- [ ] 8x8 font ROM for digits and capital letters; draw SCORE, HIGH SCORE and lives in the right-hand panel
- [ ] Count remaining pellets; at 0, reload the maze

Done when: dots vanish as you eat them, the score is right, and clearing the board starts a new level.

Phase 7 — Ghosts, first version

- [ ] Ghost sprites: 2 body frames plus eyes that look in the direction of travel; one shared shape recoloured per ghost by palette
- [ ] Movement rule: at each tile centre, pick from the open directions (never reversing) the one closest to a target tile, using squared distance so there's no square root
- [ ] For now every ghost targets Pac-Man's tile
- [ ] Ghost house: release ghosts one at a time on a timer
- [ ] Same tile as a ghost = death; 3 lives, death animation, reset positions, GAME OVER text

Done when: four ghosts leave the house and chase you, and getting caught costs a life.

Phase 8 — Arcade ghost behaviour

- [ ] Personal targets: red targets Pac-Man's tile; pink targets 4 tiles ahead of him; cyan doubles the vector from red to 2 tiles ahead of Pac-Man; orange chases when more than 8 tiles away, otherwise heads to its corner
- [ ] Scatter/chase timer: ghosts alternate between heading to their own corners and chasing, reversing direction on each switch
- [ ] Power pellets: frightened mode (blue, slower, random turns from an LFSR, flashing white near the end); eaten ghosts score 200/400/800/1600 and return home as eyes
- [ ] Per-level speed table and shorter frightened time as levels rise

Done when: each ghost visibly behaves differently, and power pellets turn the tables.

Phase 9 — Game flow and sound

- [ ] Top-level state machine: title screen → READY! → playing → death → level clear (maze flashes) → game over → title
- [ ] Keep the high score in a register for the session

- [ ] Sound: square-wave tones via PWM on a free pin into a piezo; a short "wakka" per pellet, a siren while playing, a falling tone on death

Done when: a complete game plays from power-on to game over and back to the title.

Phase 10 — Finish

- [ ] Clean timing (WNS positive), no synthesis warnings you can't explain
- [ ] Write to flash so it boots standalone
- [ ] Record a video and write up the architecture; it's a strong project to show on a CV

Debugging tips and common pitfalls

Most FPGA video bugs fall into a handful of patterns; match the symptom first.

Symptom	Likely cause	Fix
Monitor says "No signal" or "Out of range"	Wrong totals (not 800 x 525), wrong sync polarity, or pixel clock not running	Check counts in simulation; syncs are active-low in this mode; check the Clocking Wizard's `locked` output
Colours washed out or picture too bright	RGB not forced to 0 during blanking	Gate RGB with `video_on`; monitors calibrate black from the blanking interval
Sprites or tiles shifted 1-2 pixels, garbage at left edge	Sync signals not delayed to match the RAM pipeline	Delay `hsync`, `vsync`, `video_on` by the same number of cycles as the pixel data
Pac-Man flickers or tears	Positions change while the beam is drawing him	Update game state only on `frame_tick`, during vertical blanking
Button presses register twice or randomly	Raw mechanical input used directly	Synchronise with 2 flip-flops, then debounce
Works in simulation, not on hardware	Unconstrained pin, latch, or negative timing slack	Read synthesis warnings and the timing summary; check the XDC port names match

Symptom	Likely cause	Fix
Sprite ROM used thousands of LUTs	ROM not inferred as block RAM	Use a clocked (synchronous) read and `$readmemh`; check the Utilization report

Rules that prevent most of those

- **One clock.** Never make a slow clock with a counter and use it in `posedge`. Use the 25.175 MHz clock everywhere, with enable pulses like `frame_tick` for slower work.
- **No latches.** In every `always @*` block, assign every output in every branch; a default assignment at the top of the block does it.
- **`<=` in clocked blocks, `=` in combinational blocks.** Mixing them causes simulation and hardware to disagree.
- **Long maths gets pipelined.** Ghost distance calculations (two multiplies and a compare per direction) can be done one direction per clock cycle over many cycles, since you have the whole vertical blank (about 36,000 cycles) to finish.

Debugging on the real board

- **LEDs** are the quickest probe: tie one to `frame_tick` divided down, or to a state machine bit.
- **Integrated Logic Analyzer (ILA).** After synthesis, Set Up Debug lets you mark internal signals and watch them live through the USB cable in Hardware Manager, with triggers. Use it for game-logic bugs like a ghost stuck at a junction.
- **Debug overlay.** Add a switchable mode that draws Pac-Man's tile coordinates or a ghost's target tile on screen; the monitor becomes your scope.

Reference to keep open

Jamey Pittman's *The Pac-Man Dossier* is the standard write-up of the arcade game's ghost targeting, timers and speed tables. Use it for Phase 8.